

Megapot V2: Mitigation Review & Buy Helpers Audit

Joint audit report

[montecristo](#) | [hgrano](#) | [0xnev](#)

Dec 24, 2025

Contents

Summary	5
Scope	6
Medium Severity	7
M-01 It is still possible to DoS decreases to lpPoolCap	7
Target	7
Finding Description and Impact	7
Recommended Mitigation Steps	8
Resolution	8
M-02 Subscription execution will incorrectly cancel subscriptions created after price change	8
Target	8
Finding Description and Impact	8
Recommended Mitigation Steps	9
Proof of Concept	10
Resolution	10
Low Severity	11
L-01 Missing event for LP pool cap update	11
Target	11
Finding Description and Impact	11
Recommended Mitigation Steps	11
Resolution	11
L-02 Mitigation of F-59 can be better optimized	11
Target	11
Finding Description and Impact	12
Recommended Mitigation Steps	13
Resolution	15

L-03	Minimum ticket count can be set to 0	15
	Target	15
	Finding Description and Impact	15
	Recommended Mitigation Steps	16
	Resolution	16
L-04	Black-listed users will cause reverts when cancelling Batch purchases	16
	Target	16
	Finding Description and Impact	16
	Recommended Mitigation Steps	16
	Resolution	16
L-05	It is possible to cause temporary DoS for auto-subscriptions	16
	Target	16
	Finding Description and Impact	16
	Recommended Mitigation Steps	17
	Resolution	17
Informational		17
I-01	Arbitrary relay data allowed during bridging	17
	Target	17
	Finding Description and Impact	17
	Recommended Mitigation Steps	18
	Resolution	18
I-02	Incorrect parameter name in JackpotLPManager interface	18
	Target	18
	Finding Description and Impact	18
	Recommended Mitigation Steps	19
	Resolution	19
I-03	Redundant bridgeDetails.approveTo and bridgeDetails.to parameters in Jackpot-BridgeManager	19
	Target	19

Finding Description and Impact	19
Recommended Mitigation Steps	19
Resolution	19
Disclaimer	19
Appendix: Fuzz testing	20
Fuzz testing the gas optimization	20
TicketComboTracker soundness fuzz testing	21

Summary

Timeline Dec 2025 (6 audit days)
Languages Solidity
Total Issues 10 (7 resolved, 3 acknowledged)

Medium Severity Issues 2 (1 resolved, 1 acknowledged)
Low Severity Issues 5 (3 resolved, 2 acknowledged)
Informational issues 3 (3 resolved)

Scope

The scope of the audit was split into two phases. In the first phase, the `coordinationlabs/megapot-v2-contracts` repository at commit [52468c7](#) was audited with the following files being in scope:

```
contracts
├── GuaranteedMinimumPayoutCalculator.sol
├── interfaces
│   ├── IJackpotLPManager.sol
│   ├── IJackpot.sol
│   ├── IJackpotTicketNFT.sol
│   ├── IPayoutCalculator.sol
│   └── IScaledEntropyProvider.sol
├── JackpotBridgeManager.sol
├── JackpotLPManager.sol
├── Jackpot.sol
├── JackpotTicketNFT.sol
├── lib
│   ├── AllowedManager.sol
│   ├── Combinations.sol
│   ├── FisherYatesWithRejection.sol
│   ├── JackpotErrors.sol
│   ├── TicketComboTrackerOptimized.sol
│   ├── TicketComboTracker.sol
│   └── UintCasts.sol
└── ScaledEntropyProvider.sol
```

The code had been previously audited through a Code4rena contest: [2025-11-megapot](#). The primary focus of this phase was reviewing mitigations applied by the Megapot team in response to the Code4rena findings. The scope was restricted only to the above files.

In the second phase of the audit, the same repository was audited at commit [9b61e56](#) with only the files relevant to the buy helpers being in scope:

```
contracts
├── BatchPurchaseFacilitator.sol
├── interfaces
│   └── IBatchPurchaseFacilitator.sol
├── JackpotAutoSubscription.sol
├── JackpotRandomTicketBuyer.sol
├── lib
│   ├── AllowedManager.sol
│   ├── BuyTicketsHelpers.sol
│   └── TicketPicker.sol
```

For simplicity, all findings for both phases are collated into this single report.

Medium Severity

M-01 It is still possible to DoS decreases to lpPoolCap

Target

JackpotLPManager.sol:434-448

Finding Description and Impact

The issue described in F-112 is not mitigated. Consider that an attacker could make LP deposits up until we reach the current lpPoolCap. If this happens then there is no way for the admin to decrease lpPoolCap. As shown by the below snippets, this will cause a revert during JackpotLPManager.setLPPoolCap:

Jackpot.sol:1557-1563:

```

1557     function _validateAndUpdateLpPoolCap(uint256 _normalBallMax) internal {
1558         uint256 lpSoftCap = _calculateLpPoolSoftCap(_normalBallMax);
1559
1560         // We need to make sure that the lpPoolCap is not greater than the
1561         // governance pool cap
1562         uint256 lpPoolCap = Math.min(lpSoftCap, governancePoolCap);
1563         jackpotLPManager.setLPPoolCap(currentDrawingId, lpSoftCap, lpPoolCap);
1564     }

```

Jackpot.sol:1580-1587:

```

1580     function _calculateLpPoolSoftCap(uint256 _normalBallMax) internal view
1581     returns (uint256) {
1582         // We use bonusballSoftCap in order to ensure that there is sufficient
1583         // buffer before we hit the hard cap. Once the soft cap is hit
1584         // deposits cannot be made but the lpPool can continue to grow from
1585         // profits with no adverse effects to LPs. Once the hard cap is hit
1586         // the lpPool can still grow from profits but LPs will see deterioration
1587         // in collected edge per ticket.
1588         uint256 maxAllowableTickets = Combinations.choose(_normalBallMax,
1589             NORMAL_BALL_COUNT) * (bonusballSoftCap);
1590         uint256 maxPrizePool = maxAllowableTickets * ticketPrice * (PRECISE_UNIT
1591             - lpEdgeTarget) / PRECISE_UNIT;
1592         return maxPrizePool * PRECISE_UNIT / (PRECISE_UNIT - reserveRatio);
1593     }

```

Then, in JackpotLPManager we get a revert if lpSoftCap <= nextDrawingLpPool, which is possible due to the attacker increasing pendingDeposits:

JackpotLPManager.sol:434-448:

```

434     function setLPPoolCap(uint256 _drawingId, uint256 _lpSoftCap, uint256
435         _lpPoolCap) external onlyJackpot() {
436         uint256 nextDrawingLpPool = _calculateNextDrawingLpPool(lpDrawingState[
437             _drawingId]);
438
439         // We need to ensure that soft cap is not exceeded by the next drawing's
440         // lp pool total regardless of
441         // what the actual lpPoolCap is set to (since governance cap is more of
442         // a soft cap)

```

```

440     if (_lpSoftCap <= nextDrawingLpPool) revert InvalidLPSoftCap();
441
442
443     // In cases where governance is capping the pool size and remains
444     // unchanged we can skip the update
445     // and checks. If governance cap is changed we need to ensure that the
446     // new cap is not exceeded by
447     // the next drawing's lp pool total
448     if (_lpPoolCap != lpPoolCap) {
449         if (_lpPoolCap < nextDrawingLpPool) revert InvalidLPPoolCap();
450         lpPoolCap = _lpPoolCap;
451     }
452 }

```

This will block the following changes:

- Decreasing governancePoolCap
- Decreasing reserveRatio
- Decreasing ticketPrice
- Decreasing normalBallMax
- Increasing lpEdgeTarget

Recommended Mitigation Steps

Instead of reverting if nextDrawingLpPool is in excess of the new lpPoolCap, consider permitting the state update anyway: the impact is that new deposits cannot be made until enough LPs have withdrawn.

Resolution

Megapot: Acknowledged.

M-02 Subscription execution will incorrectly cancel subscriptions created after price change

Target

[JackpotAutoSubscription.sol:426-431](#)

Finding Description and Impact

When a subscription is created, if the ticket price has changed, subscription start is set to the next drawing:

[JackpotAutoSubscription.sol:232-244:](#)

```

232 uint256 currentTicketPrice = drawingState.ticketPrice;
233 @> uint64 drawingStart = currentTicketPrice == globalTicketPrice ? uint64(currentDrawingId -
234     1) : uint64(currentDrawingId);

```



```

235 subscriptions[_recipient] = Subscription({
236     remainingUSDC: uint64(totalCost),
237 @> lastExecutedDrawing: drawingStart,
238     subscribedTicketPrice: uint64(globalTicketPrice),
239     dynamicTicketCount: uint64(_dynamicTicketCount),
240     referrers: _referrers,
241     referralSplit: _referralSplit
242 });

```

This is to skip subscription execution until the next drawing, when the new ticket price gets effective.

However, if the subscription is included in a batch execution in the current drawing, it will be cancelled due to price mismatch:

[JackpotAutoSubscription.sol:414-430](#):

```

414 function _determineExecutionAction(
415     address _recipient,
416     uint256 _currentDrawingId,
417     uint256 _ticketPrice,
418     uint256 _maxReferrers
419 )
420     internal
421     view
422     returns (ExecutionAction)
423 {
424     Subscription memory subscription = subscriptions[_recipient];
425     if (subscription.remainingUSDC == 0) return ExecutionAction.
        SKIP_NO_ACTIVE_SUBSCRIPTION;
426 @> if (
427 @>
        subscription.subscribedTicketPrice != _ticketPrice && subscription.subscribedTicketPrice != 0
428 @> ) return ExecutionAction.CANCEL_PRICE_CHANGE;
429     if (subscription.referrers.length > _maxReferrers) return ExecutionAction.
        CANCEL_TOO_MANY_REFERRERS;
430 @>
        if (subscription.lastExecutedDrawing == _currentDrawingId) return ExecutionAction.SKIP_ALREADY_EXECUTED;

```

The original intention was to trigger L430 and skip the execution until the next drawing.

However, since the price check (L426-L428) comes before the drawing check (L430), the subscription is cancelled instead of being skipped.

Recommended Mitigation Steps

The following diff will fix the issue:

```

diff --git a/contracts/JackpotAutoSubscription.sol b/contracts/
    JackpotAutoSubscription.sol
index daf3efb..e9fbeb8 100644
--- a/contracts/JackpotAutoSubscription.sol
+++ b/contracts/JackpotAutoSubscription.sol
@@ -423,11 +423,11 @@ contract JackpotAutoSubscription is Ownable2Step,
    ReentrancyGuardTransient, Allow
    {
        Subscription memory subscription = subscriptions[_recipient];
        if (subscription.remainingUSDC == 0) return ExecutionAction.
            SKIP_NO_ACTIVE_SUBSCRIPTION;

```

```

+         if (subscription.lastExecutedDrawing == _currentDrawingId) return
ExecutionAction.SKIP_ALREADY_EXECUTED;
        if (
            subscription.subscribedTicketPrice != _ticketPrice && subscription.
            subscribedTicketPrice != 0
        ) return ExecutionAction.CANCEL_PRICE_CHANGE;
        if (subscription.referrers.length > _maxReferrers) return
ExecutionAction.CANCEL_TOO_MANY_REFERRERS;
-         if (subscription.lastExecutedDrawing == _currentDrawingId) return
ExecutionAction.SKIP_ALREADY_EXECUTED;

        uint256 totalTicketCount = staticTickets[_recipient].length +
            subscription.dynamicTicketCount;
        uint256 totalCost = totalTicketCount * _ticketPrice;

```

Proof of Concept

Apply the following diff and run `yarn hardhat test ./test/jackpotAutoSubscription.spec.ts`:

```

diff --git a/test/jackpotAutoSubscription.spec.ts b/test/jackpotAutoSubscription
.spec.ts
index 75b94de..6eafc5a 100644
--- a/test/jackpotAutoSubscription.spec.ts
+++ b/test/jackpotAutoSubscription.spec.ts
@@ -229,6 +229,22 @@ describe("JackpotAutoSubscription", () => {
    usdc(2)
    );
  });
+
+  it.only("execution will cancel subscription", async () => {
+    await subject();
+    const subjectSubscriptions = [subjectRecipient];
+    await expect(
+      jackpotAutoSubscription
+        .connect(keeper.wallet)
+        .executeSubscriptions(subjectSubscriptions),
+    )
+      .to.emit(jackpotAutoSubscription, "SubscriptionCancelled")
+      .withArgs(
+        subjectRecipient,
+        ExecutionAction.CANCEL_PRICE_CHANGE,
+        usdc(40),
+      );
+  });
+});

describe("when the referrer count is greater than the current global max
referrer count", () => {

```

Resolution

Megaprot: Fixed by [6a208434e04d1b01048979afadedecfb196d0f3e](#).

Audit team: Verified.

Low Severity

L-01 Missing event for LP pool cap update

Target

[JackpotLPManager.sol:L446](#)

Finding Description and Impact

When LP pool cap is updated, there is no event emitted:

[JackpotLPManager.sol:434-448](#):

```

434 function setLPPoolCap(uint256 _drawingId, uint256 _lpSoftCap, uint256 _lpPoolCap
    ) external onlyJackpot() {
435     uint256 nextDrawingLpPool = _calculateNextDrawingLpPool(lpDrawingState[
        _drawingId]);
436
437     // We need to ensure that soft cap is not exceeded by the next drawing's lp
        pool total regardless of
438     // what the actual lpPoolCap is set to (since governance cap is more of a
        soft cap)
439     if (_lpSoftCap <= nextDrawingLpPool) revert InvalidLPSoftCap();
440
441     // In cases where governance is capping the pool size and remains unchanged
        we can skip the update
442     // and checks. If governance cap is changed we need to ensure that the new
        cap is not exceeded by
443     // the next drawing's lp pool total
444     if (_lpPoolCap != lpPoolCap) {
445         if (_lpPoolCap < nextDrawingLpPool) revert InvalidLPPoolCap();
446 @>         lpPoolCap = _lpPoolCap;
447     }
448 }
```

Recommended Mitigation Steps

Add LpPoolCapUpdated event so that offchain components can be notified on the important config update.

Resolution

Megapot: Fixed by [3e728c0105c481ba9b6f55e918fb6f7298479696](#).

Audit team: Verified.

L-02 Mitigation of F-59 can be better optimized

Target

[Jackpot.sol:1618-1619](#)

Finding Description and Impact

In order to mitigate F-59 the code implements a hardcap on the new bonus ball.

Jackpot.sol:1607-1623:

```

1607 uint256 combosPerBonusball = Combinations.choose(normalBallMax,
1608           NORMAL_BALL_COUNT);
1609
1610 uint256 minNumberTickets = newPrizePool * PRECISE_UNIT / ((PRECISE_UNIT -
1611           lpEdgeTarget) * ticketPrice);
1612
1613 // Calculate bonusball without capping it based on bonusballHardCap
1614 uint8 uncappedBonusball = uint8(Math.max(bonusballMin, Math.ceilDiv(
1615           minNumberTickets, combosPerBonusball)));
1616
1617 // Cap the bonusball based on bonusballHardCap - we use the hard cap here
1618 // because we cannot let the bonusball go above
1619 // the hard cap otherwise we could lock the next drawing settlement due to
1620 // exceeding gas limits or over-packing the ticket
1621 // bit vector. Note that we validate that the bonusballHardCap + normalBallMax
1622 // is less than 255 in _validateBallSettings. It
1623 // is up to governance to set a hard cap that does not lead to exceeding gas
1624 // limits.
1625 uint8 newBonusball;
1626 if (uncappedBonusball >= bonusballHardCap) {
1627     newBonusball = bonusballHardCap;
1628     emit BonusballHardCapHit(currentDrawingId, uncappedBonusball,
1629         bonusballHardCap);
1630 } else {
1631     newBonusball = uncappedBonusball;
1632 }

```

Firstly, there is an issue (acknowledged by the Megapot team [here](#)) where by LP earnings will be degraded (and can in fact go negative):

Consider a situation where we reach the `bonusballHardCap` and over time, due to LP earnings, the prize pool increases. Eventually, we could have a situation where the prize pool is large but the number of possible tickets is insufficient to protect the intended edge LPs should have. We can calculate (assuming no duplicate tickets), that a gambler purchasing one of each distinct available ticket will make this profit:

```

gamblerProfit = prizePool - totalTickets * ticketPrice
               = prizePool - choose(normalBallMax, 5) * bonusBallmax *
                 ticketPrice

```

If we solve for `gamblerProfit > 0` we get:

```

prizePool > choose(normalBallMax, 5) * bonusBallmax * ticketPrice

```

Taking an example where:

- `bonusBallmax = bonusballHardCap = 100`
- `ticketPrice = $1`
- `normalBallMax = 30`

We get:

```
prizePool > $14 million
```

This prize pool is not likely to arise in practice (considering the admin would likely increase `normalBallMax` before it gets close to this). Therefore the likelihood is low. The impact is high because gamblers could drain funds at the expense of LPs.

Not only is there a potential for losses for LPs due to this mitigation, there is also an alternative gas optimization (see below) which avoids the problem entirely.

Recommended Mitigation Steps

There is an alternative mitigation to F-59 which does not require the `bonusballHardCap`. Within `TicketComboTracker::_countSubsetMatches`, the following summation is the source of possible DoS.

[TicketComboTracker.sol:162-175](#):

```

162 for (uint8 i = 1; i <= _tracker.bonusballMax; i++) {
163     for (uint8 k = 1; k <= _tracker.normalTiers; k++) {
164         uint256[] memory subsets = subsetsArr[k-1];
165         for (uint256 l = 0; l < subsets.length; l++) {
166             if (i == _bonusball) {
167                 matches[(k*2)+1] += _tracker.comboCounts[i][subsets[l]].count;
168                 dupMatches[k*2+1] += _tracker.comboCounts[i][subsets[l]].
                    dupCount;
169             } else {
170                 matches[k*2] += _tracker.comboCounts[i][subsets[l]].count;
171                 dupMatches[k*2] += _tracker.comboCounts[i][subsets[l]].dupCount;
172             }
173         }
174     }

```

We can interpret the `matches` array as:

```

matches[k*2] --> count of (non-duplicate) tickets with a length k subset match,
                but no bonus ball match
matches[k*2+1] --> count of (non-duplicate) tickets with a length k subset match
                , with bonus ball match

```

`dupMatches` follows the same principle as `matches` but counts only duplicate tickets.

Observe that if we introduce a new counter in storage for *all* (non-duplicate) tickets which include `subset[1]` (regardless of bonus ball selection):

```
_tracker.comboCounts[0][subsets[1]].count
```

then the contribution to `matches[k*2]` from subset 1 is just the above total counter minus those tickets which do have the bonus ball match.

Similarly, we can also track a count of all duplicate tickets which include `subset[1]` regardless of bonus ball in:

```
_tracker.comboCounts[0][subsets[1]].dupCount
```

These additional counters then remove the need for the outer loop from 1 to `bonusBallMax`. The `TicketComboTracker::_countSubsetMatches` nested loop can be re-written as:

```

for (uint8 k = 1; k <= _tracker.normalTiers; k++) {
    uint256[] memory subsets = subsetsArr[k-1];
    for (uint256 l = 0; l < subsets.length; l++) {
        uint256 totalCount = _tracker.comboCounts[0][subsets[l]].count;
        uint256 totalDupCount = _tracker.comboCounts[0][subsets[l]].dupCount;
        uint256 bonusBallMatchCount = _tracker.comboCounts[_bonusball][subsets[l]].count;
        uint256 bonusBallDupCount = _tracker.comboCounts[_bonusball][subsets[l]].dupCount;

        matches[(k*2)+1] += bonusBallMatchCount;
        dupMatches[k*2+1] += bonusBallDupCount;
        matches[(k*2)] += totalCount - bonusBallMatchCount;
        dupMatches[k*2] += totalDupCount - bonusBallDupCount;
    }
}

```

It is straightforward to update the total counters when a ticket is purchased:

```

index 7f04c29..0ba8b9d 100644
--- a/contracts/lib/TicketComboTracker.sol
+++ b/contracts/lib/TicketComboTracker.sol
@@ -112,36 +112,38 @@ library TicketComboTracker {
    function insert(
        Tracker storage _tracker,
        uint8[] memory _normalBalls,
        uint8 _bonusball
    )
        internal
        returns (uint256 ticketNumbers, bool isDup)
    {
        require(_normalBalls.length == _tracker.normalTiers, "Invalid pick length");
        uint256 set = toNormalsBitVector(_normalBalls, _tracker.normalMax);
        // Iterate over all tier combos and store the combo counts
        isDup = _tracker.comboCounts[_bonusball][set].count > 0;
        for (uint8 i = 1; i <= _tracker.normalTiers; i++) {
            uint256[] memory subsets = Combinations.generateSubsets(set, i);
            for (uint256 j = 0; j < subsets.length; j++) {
                if (isDup) {
                    _tracker.comboCounts[_bonusball][subsets[j]].dupCount++;
+                    _tracker.comboCounts[0][subsets[j]].dupCount++;
                } else {
                    _tracker.comboCounts[_bonusball][subsets[j]].count++;
+                    _tracker.comboCounts[0][subsets[j]].count++;
                }
            }
        }
    }
}

```

This mitigation comes at a slight gas cost increase for ticket purchases but removes any chance of an out-of-gas issue due to a high bonus ball max. This way, the `bonusballHardCap` could be removed. However, we still run a risk of F-81 occurring - i.e. the sum of `bonusballMax` and `normalBallMax` could theoretically exceed 255 for an unusually large prize pool. To remedy this, logic could be added to `Jackpot::scaledEntropyCallback` so that it automatically puts the contract into the emergency mode if this invariant does not hold. After this, ticket holders and LPs could still claim their fair share of funds, including winnings from this final drawing.

Fuzz testing was conducted to check correctness of this proposed mitigation. Please refer to the appendix for further details

Resolution

Megapot: Acknowledged. The increased gas usage on ticket purchases as a result of the proposed mitigation is considered too high a cost for users to bear:

Statement by Megapot team:

This is more efficient for settlement I agree, however it doubles the cost of storing tickets during purchase. This is ultimately one of the big tensions for this protocol is making the gas cost for buying tickets reasonable while also making the jackpot able to settle within one transaction. In this case I think the trade-off to user experience is too great. We are already pretty limited in the amount of tickets that can be bought per txn and this solution would constrain us even further. Given the v low likelihood of issues arising here I'm inclined to keep the soft cap and hard cap dynamic.

To put some numbers to it - in the worst case scenario the new subsets are being written from 0 which is about 22k extra gas per combination of which there are 31 per ticket. This results in an additional 680k gas per ticket in the worst case. Best case is that all of the slots are warm - at 5k per combination that is an addition 110k gas per ticket. The reality is that it will be somewhere in between maybe closer to the middle since this aggregates across bonusballs.

Given the per txn gas limits on Base around 25M you can see how this starts drastically reducing the amount of tickets that can be bought with one purchase. While we have some execution strategies to reduce the friction in the product it would require significantly more aggregate transactions to purchase the same amount of tickets which we already are worried about the effects on the network during high volume periods.

Audit team: Existing mitigation already implemented by Megapot team is acceptable.

L-03 Minimum ticket count can be set to 0

Target

[BatchPurchaseFacilitator.sol:329-331](#)

Finding Description and Impact

BatchPurchaseFacilitator enforces that purchases contain at least one ticket:

[BatchPurchaseFacilitator.sol:168-170:](#)

```

168 @> // Purchases must be at least one ticket
169     uint64 ticketCount = uint64(_dynamicTicketCount) + uint64(_userStaticTickets
    .length);
170 @> if (ticketCount < minimumTicketCount) revert InvalidTicketCount();

```

However, it's possible to set `minimumTicketCount` to 0:

[BatchPurchaseFacilitator.sol:329-331:](#)

```

329 function setMinimumTicketCount(uint256 _minimumTicketCount) external onlyOwner {
330 @> minimumTicketCount = _minimumTicketCount;
331 }

```

In this case, it is possible to create an empty order (i.e. ticket count = 0).

Recommended Mitigation Steps

Check if new `minimumTicketCount` is greater than 0.

Resolution

Megaplot: Fixed by [76fb43deff5d673caee44801d493cad0eb0c801e](#).

Audit team: Verified.

L-04 Black-listed users will cause reverts when cancelling Batch purchases

Target

[BatchPurchaseFacilitator.sol:388](#)

Finding Description and Impact

On `BatchPurchaseFacilitator.sol:388`, if the `recipient` is blacklisted by USDC then this will cause a revert. This will affect calls to `BatchPurchaseFacilitator::executeBatchOrder`, which may try to cancel the order. The same problem exists on `JackpotAutoSubscription:494`. This will cause reverts of `JackpotAutoSubscription::executeSubscriptions`, in the case where orders need to be cancelled e.g. due to price changes.

Recommended Mitigation Steps

Consider using a try/catch around the `safeTransfer` call so that if the transfer fails, the transaction isn't reverted. Alternatively, switch to a different design where users would need to withdraw funds themselves on refunds, rather than directly transferring to them.

Resolution

Megaplot: Fixed by [76fb43deff5d673caee44801d493cad0eb0c801e](#).

Audit team: Verified.

L-05 It is possible to cause temporary DoS for auto-subscriptions

Target

[JackpotAutoSubscription.sol:317-318](#)

Finding Description and Impact

Consider a scenario where Alice, a whale gambler, has auto-subscribed using `JackpotAutoSubscription` for a large daily ticket count. Bob, another gambler, sees this and wants to increase his potential share of the prize pool by owning a larger proportion of the total tickets available. Bob calls

`BatchPurchaseFacilitator::createBatchOrder` for `recipient = Alice` for the `minimumTicketCount` (much lower than the number of tickets Alice wanted). Now when the keeper calls `JackpotAutoSubscription::executeSubscriptions` the determined execution action will be `ExecutionAction.SKIP_ACTIVE_BATCH_ORDER` on `JackpotAutoSubscription.sol:437`. Alice's order is then skipped for the current drawing - unless the keeper subsequently calls `executeBatchOrder` for `recipient = Alice` followed by calling `executeSubscriptions`. If we assume the keeper calls `executeSubscriptions` at roughly the same time every day, then potentially an attacker could observe this and call `createBatchOrder` just before this time. This is unlikely to occur in practice due to these timing difficulties and the cost to Bob, hence the severity rating is low.

Recommended Mitigation Steps

Consider potential restrictions on the `recipient` argument of `BatchPurchaseFacilitator::createBatchOrder`. This could be restricted to only `msg.sender` or authorized addresses such as the `JackpotAutoSubscription`.

Resolution

Megaplot: Acknowledged. [Response by Megaplot team:](#)

Acknowledged - yes this is a potential issue. As you mention it is also dependent on the execution algorithm the keepers use. Realistically these executions are likely to take place over the course of the day leading up to the drawing. In order to meet our on-ramping requirements (on-ramp -> transactions, instead of on-ramp -> deposit -> txn) we need to build the flexibility to have addresses create orders on behalf of others (and it doesn't seem we can whitelist certain ones). Certainly this is something we will monitor for potential abuse - ultimately the likely outcome here is that the attacker ends up gifting even more tickets to the whale.

Informational

I-01 Arbitrary relay data allowed during bridging

Target

[JackpotBridgeManager.sol:361](#)

Finding Description and Impact

Notice in `JackpotBridgeManager.claimWinnings`, the `_bridgeDetails.data` data parameter is arbitrary. If we look at the intended [RelayDepository](#) contract to be used, this could be slightly risky, although it would require a malicious/wrong signature provided for the winning ticket owner.

1. The `depositNative` function can be called which will simply revert, since jackpot rewards are always denoted in USDC
2. In `depositERC20`, if the `depositor` input encoded within the `_bridgeDetails.data` is `address(0)`, the `depositorAddress` will be attributed to the bridge itself within the `de-`

[positErc20 function here](#) as it is the caller of the `depositErc20` function, which can possibly cause fund lock (although requires a reckless wrong signature). Funds will need to be redirected on the relay protocol's allocator side.

Recommended Mitigation Steps

The data input should be preconstructed and call towards only the [depositErc20 function](#) with the following:

- **depositor:** Ticket owner or recipient designated by ticket owner performing claim
- **token:** Hardcoded to USDC
- **amount:** `_claimedAmount`
- **id:** A tracked unique nonce value

Resolution

Megaplot: Fixed with [6cfa82d0507fac85ffb2bfd6564dc6f32472e465](#).

Audit team: Verified.

I-02 Incorrect parameter name in JackpotLPManager interface

Target

[IJackpotLPManager.sol:30](#)

Finding Description and Impact

`IJackpotLPManager` has incorrect naming for one of its parameters:

[IJackpotLPManager.sol:30](#):

```
30 function setLPPOOLCap(uint256 _drawingId, uint256 _lpHardCap, uint256 _lpPoolCap
    ) external;
```

Actually `_lpHardCap` is meant to be LP soft cap, as we can see in the implementation:

[JackpotLPManager.sol:434](#):

```
434 function setLPPOOLCap(uint256 _drawingId, uint256 _lpSoftCap, uint256 _lpPoolCap
    ) external onlyJackpot() {
```

[Jackpot.sol:1557-1563](#):

```
1557 function _validateAndUpdateLPPOOLCap(uint256 _normalBallMax) internal {
1558   @> uint256 lpSoftCap = _calculateLPPOOLSoftCap(_normalBallMax);
1559
1560   // We need to make sure that the lpPoolCap is not greater than the
       governance pool cap
1561   uint256 lpPoolCap = Math.min(lpSoftCap, governancePoolCap);
1562   jackpotLPManager.setLPPOOLCap(currentDrawingId, lpSoftCap, lpPoolCap);
1563 }
```

Recommended Mitigation Steps

Rename `_lpHardCap` to `_lpSoftCap`.

Resolution

Megapot: Fixed by [fe5a88cf7c48af9f0283933a2e62c320086e77ec](#).

Audit team: Verified.

I-03 Redundant `bridgeDetails.approveTo` and `bridgeDetails.to` parameters in `JackpotBridgeManager`

Target

[JackpotBridgeManager.sol:353-354](#)

Finding Description and Impact

Whenever a user calls `JackpotBridgeManager::claimWinnings` they must always provide `to` and `approveTo` parameters equal to the immutable variable `relayDepository`:

[JackpotBridgeManager.sol:353-354](#):

```
353 if (_bridgeDetails.approveTo != address(relayDepository)) revert  
    InvalidBridgeApprovalAddress();  
354 if (_bridgeDetails.to != address(relayDepository)) revert InvalidBridgeToAddress  
    ();
```

Therefore these parameters could be safely removed for simplicity and gas efficiency.

Recommended Mitigation Steps

Refactor the code to remove these parameters.

Resolution

Megapot: Fixed by [539d5404b5ff9324441742126e296d8ae93ba490](#).

Audit team: Verified.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. It is a process limited by available time, resources, and expertise, aimed at identifying as many issues as possible. We cannot assure complete security after the review, nor can we promise that all issues will necessarily be discovered. To improve overall security, additional reviews, bug bounty initiatives, and continuous on-chain monitoring are highly recommended.

Appendix: Fuzz testing

Fuzz testing the gas optimization

The following Echidna Fuzz test verifies that the gas-optimized solution described in L-02 produces the same output as the original. Instructions:

1. Make a copy of `TicketComboTracker.sol` and place it at `contracts/lib/TicketComboTrackerOptimized.sol`
2. Rename the copied lib to `TicketComboTrackerOptimized` and apply the optimisation changes to it.
3. Make `FisherYatesRejection.draw` an internal function to (ensures compilation does not fail).

Add a new file `CountSubsetMatchesFuzzTest.sol` in the base directory of the project with this:

```
// SPDX-License-Identifier: unlicensed
pragma solidity ^0.8.28;

import "../contracts/lib/TicketComboTracker.sol";
import "../contracts/lib/TicketComboTrackerOptimized.sol";
import "../contracts/lib/FisherYatesWithRejection.sol";

contract CountSubsetMatchesFuzzTest {
    TicketComboTracker.Tracker private tracker;
    TicketComboTrackerOptimized.Tracker private trackerOptimized;

    uint8 private constant normalBallMax = 6;
    uint8 private constant bonusBallMax = 3;

    uint8[] winningNormals;
    uint8 winningBonus;

    constructor() {
        TicketComboTracker.init(tracker, normalBallMax, bonusBallMax, 5);
        TicketComboTrackerOptimized.init(trackerOptimized, normalBallMax,
            bonusBallMax, 5);
    }

    function insert(uint256 normalSeed, uint8 b) external {
        uint8 bonus = b % bonusBallMax + 1;
        uint8[] memory normals = toUint8Array(FisherYatesRejection.draw(1,
            normalBallMax, 5, normalSeed));

        TicketComboTracker.insert(tracker, normals, bonus);
        TicketComboTrackerOptimized.insert(trackerOptimized, normals, bonus);
    }

    function announceResult(uint256 normalSeed, uint8 b) external {
        winningBonus = b % bonusBallMax + 1;
        winningNormals = toUint8Array(FisherYatesRejection.draw(1, normalBallMax,
            5, normalSeed));
    }

    function echidna_check_matching() external view returns (bool) {
        if (winningBonus == 0) return true;
    }
}
```

```

    (uint256 winningTicket, uint256[] memory uniqueResult, uint256[] memory
      dupResult) =
      TicketComboTracker.countTierMatchesWithBonusball(tracker,
        winningNormals, winningBonus);

    (uint256 winningTicketOptimized, uint256[] memory uniqueResultOptimized,
      uint256[] memory dupResultOptimized) =
      TicketComboTrackerOptimized.countTierMatchesWithBonusball(
        trackerOptimized, winningNormals, winningBonus);

    return winningTicket == winningTicketOptimized &&
      equalArrays(uniqueResult, uniqueResultOptimized) &&
      equalArrays(dupResult, dupResultOptimized);
  }

  function toUint8Array(uint256[] memory input) private pure returns (uint8[]
    memory) {
    uint8[] memory output = new uint8[](input.length);
    for (uint256 i = 0; i < input.length; i++) {
      output[i] = uint8(input[i]);
    }
    return output;
  }

  function equalArrays(uint256[] memory a, uint256[] memory b) private pure
    returns (bool) {
    if (a.length != b.length) return false;

    for (uint i = 0; i < a.length; i++) {
      if (a[i] != b[i]) {
        return false;
      }
    }

    return true;
  }
}

```

Then run using echidna:

```

echo 'cryticArgs: ["--solc-remaps", "solady/=node_modules/solady/"]' > echidna.
yaml
echidna CountSubsetMatchesFuzzTest.sol --config echidna.yaml

```

TicketComboTracker soundness fuzz testing

The following is a fuzz test to verify TicketComboTracker counting. It randomly generates < 200 user drawings and 1 winning drawing. Counts are tracked via manual counting and via TicketComboTracker. Total count & dupe counts are compared in the end.

```

pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {LibBit} from "solady/src/utils/LibBit.sol";
import {TicketComboTrackerOptimized as TicketComboTracker} from "contracts/lib/
  TicketComboTrackerOptimized.sol";
import {FisherYatesRejection} from "contracts/lib/FisherYatesWithRejection.sol";
import {UintCasts} from "contracts/lib/UintCasts.sol";

contract TicketComboTrackerTest is Test {
  using UintCasts for uint256;

```

```

using UIntCasts for uint256[];
using TicketComboTracker for TicketComboTracker.Tracker;

uint256 constant MAX_BIT_VECTOR_SIZE = 255;
uint256 constant NORMAL_TIERS = 5;
uint256 constant TOTAL_TIER_COUNT = 12;

TicketComboTracker.Tracker tracker;
uint256[TOTAL_TIER_COUNT] uniqueResult;
uint256[TOTAL_TIER_COUNT] dupResult;
mapping(uint256 packedTicket => bool) packedTicketSet;

struct FuzzParam {
    uint8 normalMax;
    uint8 bonusballMax;
    uint256 winningRandomSeed;
    uint256[] userRandomSeeds;
}

/// forge-config: default.fuzz.runs = 20
function testTicketComboTracker(FuzzParam memory param) external {
    _boundToValidFuzzParam(param);
    tracker.init(param.normalMax, param.bonusballMax, NORMAL_TIERS.toUInt8());
    (uint8[] memory winningNormalBalls, uint8 winningBonusBall, uint256
    winningTicketExpected) =
        _getBalls(param, param.winningRandomSeed);
    uint256 totalExpected;
    for (uint256 i; i < param.userRandomSeeds.length; i++) {
        (uint8[] memory normalBalls, uint8 bonusBall, uint256 packedTicket)
        =
            _getBalls(param, param.userRandomSeeds[i]);
        tracker.insert(normalBalls, bonusBall);
        uint256 ticketTierId = _calculateTicketTierId(packedTicket,
            winningTicketExpected, param.normalMax);
        if (ticketTierId > 0) {
            totalExpected++;
            if (packedTicketSet[packedTicket]) {
                dupResult[ticketTierId]++;
            } else {
                packedTicketSet[packedTicket] = true;
                uniqueResult[ticketTierId]++;
            }
        }
    }
    (uint256 winningTicketActual, uint256[] memory uniqueResultActual,
    uint256[] memory dupResultActual) =
        tracker.countTierMatchesWithBonusball(winningNormalBalls,
            winningBonusBall);

    uint256[] memory uniqueResultExpected = new uint256[](TOTAL_TIER_COUNT);
    uint256[] memory dupResultExpected = new uint256[](TOTAL_TIER_COUNT);
    for (uint256 i; i < TOTAL_TIER_COUNT; i++) {
        uniqueResultExpected[i] = uniqueResult[i];
        dupResultExpected[i] = dupResult[i];
    }
    this.logResults(uniqueResultActual, dupResultActual);
    assertEq(winningTicketExpected, winningTicketActual);
    assertEq(uniqueResultExpected, uniqueResultActual);
    assertEq(dupResultExpected, dupResultActual);
    uint256 totalActual;
    for (uint256 i; i < uniqueResultActual.length; i++) {
        totalActual += uniqueResultActual[i] + dupResultActual[i];
    }
}

```

```

    assertEq(totalActual, totalExpected);
}

function _boundToValidFuzzParam(FuzzParam memory param) internal pure
returns (FuzzParam memory) {
    param.normalMax = bound(uint256(param.normalMax), 5, MAX_BIT_VECTOR_SIZE
        - 1).toUint8();
    param.bonusballMax = bound(uint256(param.bonusballMax), 1,
        MAX_BIT_VECTOR_SIZE - param.normalMax).toUint8();
    if (param.userRandomSeeds.length > 200) {
        uint256[] memory userRandomSeeds = param.userRandomSeeds;
        assembly {
            mstore(userRandomSeeds, 200)
        }
    }
    return param;
}

// external function call arguments are tracked in foundry call trace
function logResults(uint256[] memory _uniqueResult, uint256[] memory
    _dupResult) external {}

function _getBalls(FuzzParam memory param, uint256 randomSeed)
    internal
    pure
returns (uint8[] memory normalBalls, uint8 bonusBall, uint256
    packedTicket)
{
    normalBalls = FisherYatesRejection.draw(1, param.normalMax, 5,
        randomSeed).toUint8Array();
    bonusBall = FisherYatesRejection.draw(1, param.bonusballMax, 1,
        randomSeed)[0].toUint8();
    uint256 set = TicketComboTracker.toNormalsBitVector(normalBalls, param.
        normalMax);
    packedTicket = set | (1 << (bonusBall + param.normalMax));
}

function _calculateTicketTierId(uint256 _ticketNumbers, uint256
    _winningNumbers, uint256 _normalBallMax)
    internal
    pure
returns (uint256)
{
    uint256 matches = 0;

    // Count matching normal numbers by checking overlapping bits
    uint256 matchingBits = _ticketNumbers & _winningNumbers;

    // Count the number of set bits (matches)
    matches = LibBit.popCount(matchingBits);

    // Extract bonusball from both ticket and winning numbers
    // Bonusball is stored in the highest bits after the normal numbers
    uint256 ticketBonusball = _ticketNumbers >> (_normalBallMax + 1);
    uint256 winningBonusball = _winningNumbers >> (_normalBallMax + 1);

    uint256 bonusballMatch = (ticketBonusball == winningBonusball) ? 1 : 0;

    // We count all matches including the bonusball so if the bonusball is a
    // match we need to subtract it from matches
    return 2 * (matches - bonusballMatch) + bonusballMatch;
}
}

```